



UML Class Diagrams

Chapter 16

Applying UML and Patterns

Craig Larman

NJIT



Objective

- Create design class diagrams (DCDs).
 - Identify the classes, methods, and associations to show in a DCD.
-



Class Diagrams

- The UML has notation for showing design details in static structure.
 - Class diagrams
- The definition of design class diagrams occurs within the design phase.
 - The UML does not specifically define design class diagram.

It is a design view on SW entities, rather than an analytical view on domain concepts.

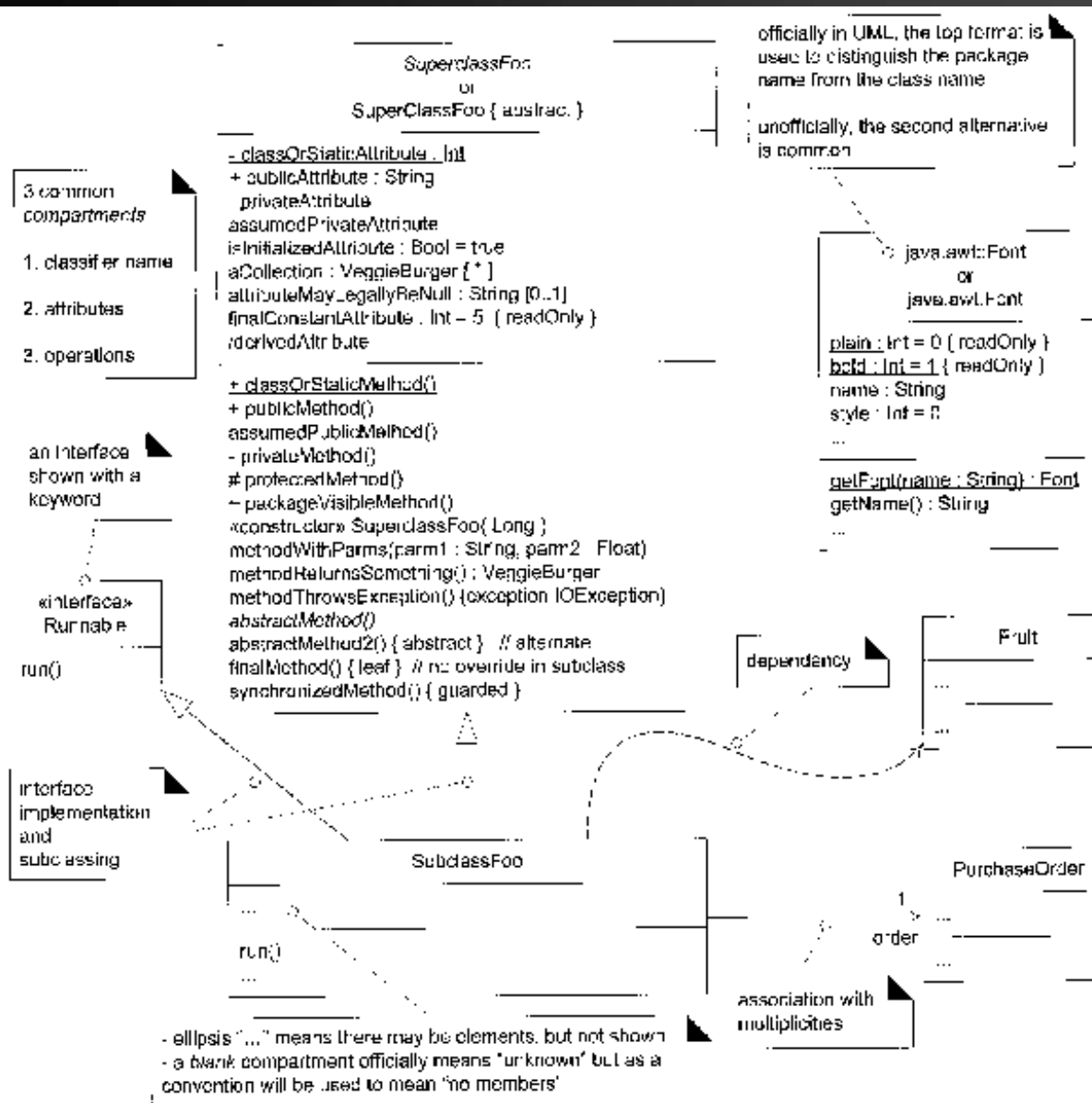


Figure 16.1 Common UML class diagram notation.

Text – Figure 16-1 DCD Summary



Design Class Diagrams

The creation of design class diagrams is dependent upon the prior creation of:

- Interaction diagrams
 - Identifies the SW classes that participate in the solution, plus the methods of classes.
 - Conceptual model
 - Adds detail to the class definitions.
-

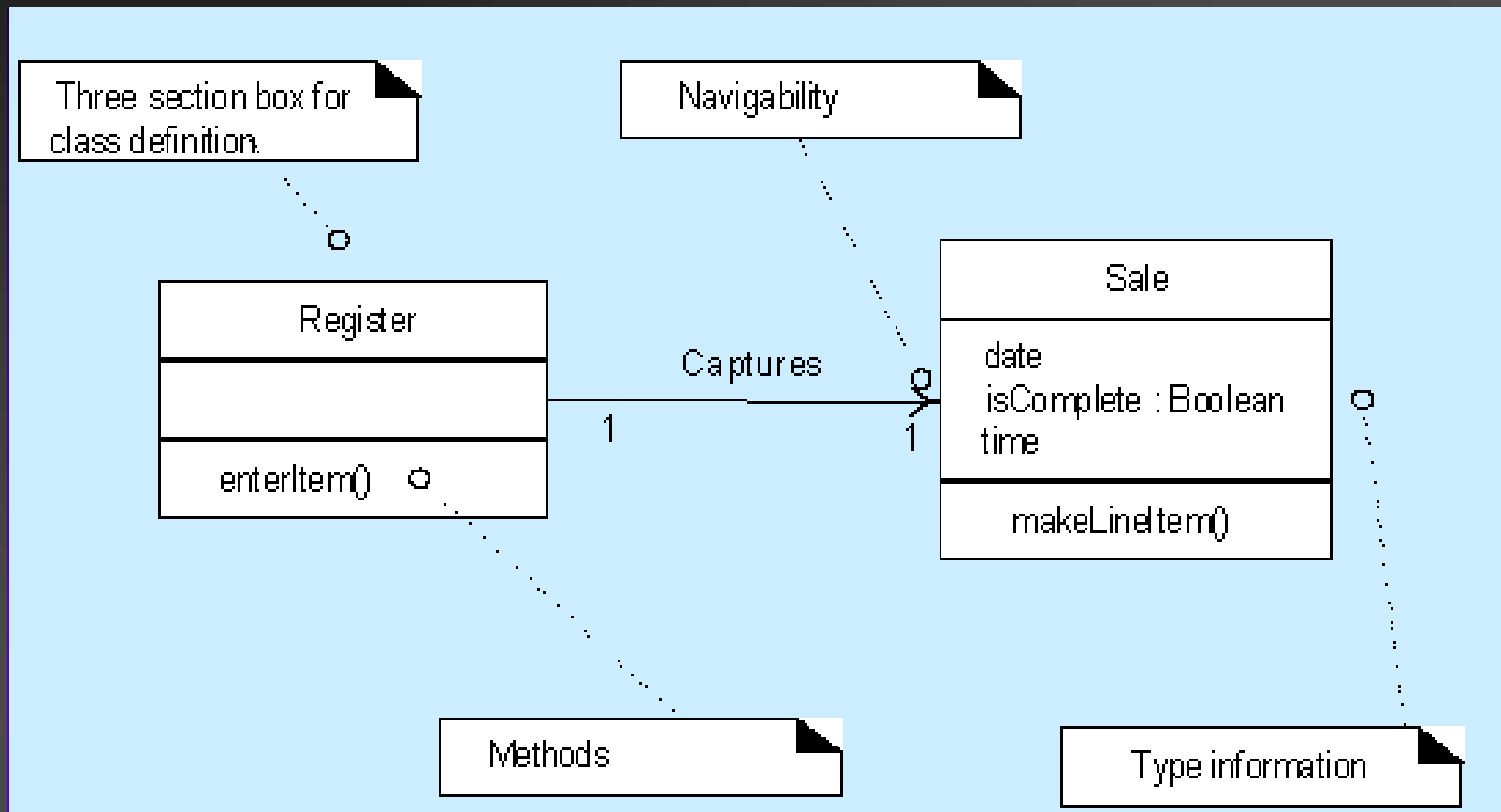


When to create DCDs.

- Although this presentation of design class diagrams follows the creation of interaction diagrams, in practice they are usually created **in parallel**.
-



Example of a DCD.





As we can see....

- A design class diagram illustrates the specifications for SW classes and interfaces.
- Typical information included:
 - Classes, associations, and attributes
 - Interfaces, with their operations and constants
 - Methods
 - Attribute type information
 - Navigability
 - Dependencies



How to make a DCD.

- Identify all the **classes** participating in the SW solution by analyzing the interaction diagrams.
- Draw them in a class diagram.
- Duplicate the **attributes** from the associated concepts in the conceptual model.
- Add **method** names by analyzing the interaction diagrams.
- Add **type** information to the attributes and methods.

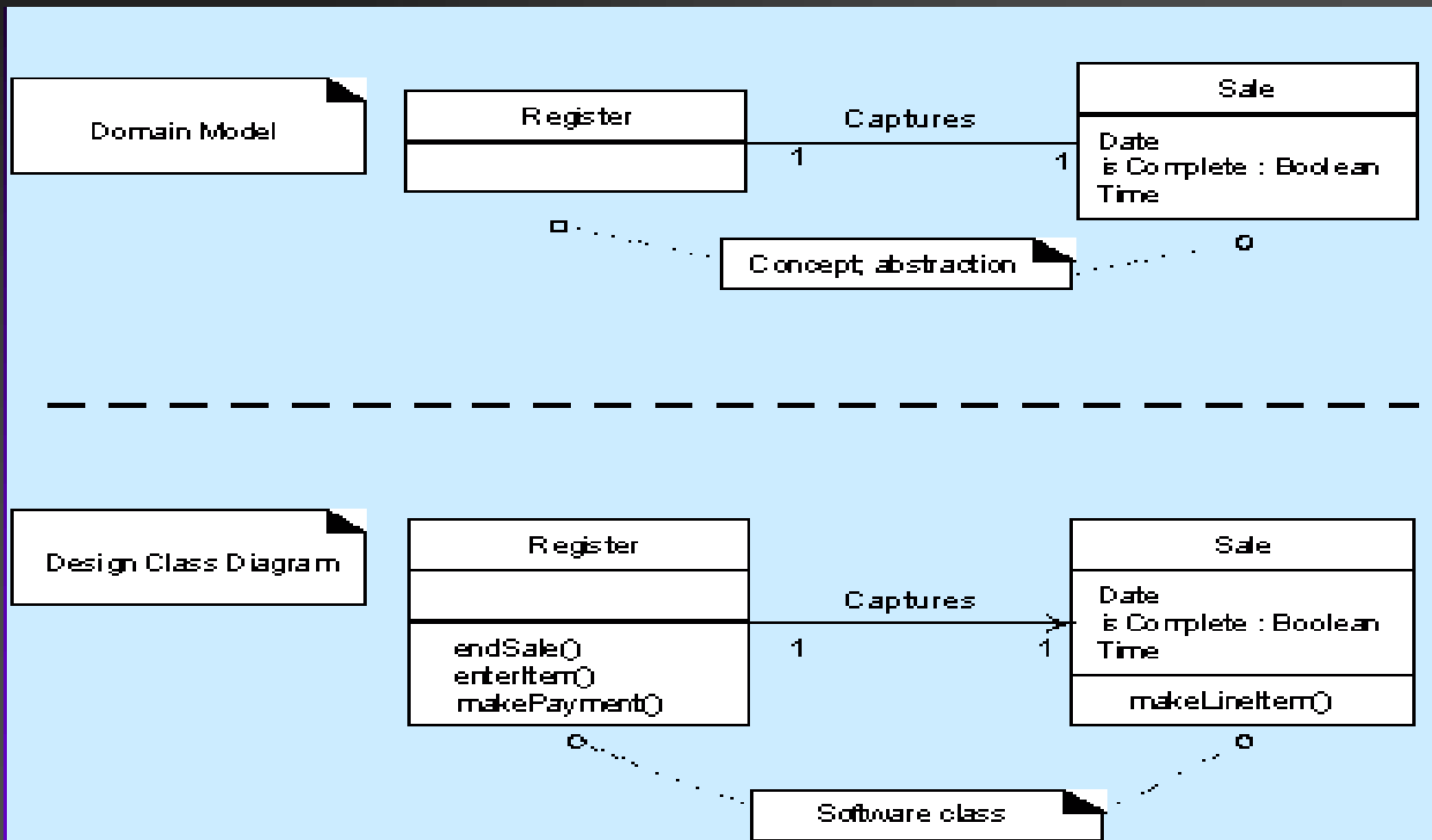


Continue...

- Add the **associations** necessary to support the required attribute visibility.
 - Add **navigability** arrows to the associations to indicate the direction of attribute visibility.
 - Add **dependency** relationship lines to indicate non-attribute visibility.
-



Domain model Vs. DCD



Creating the NextGen POS DCD.



- Identify SW classes and illustrate them.

Register

ProductCatalog

Store

Payment

Sale

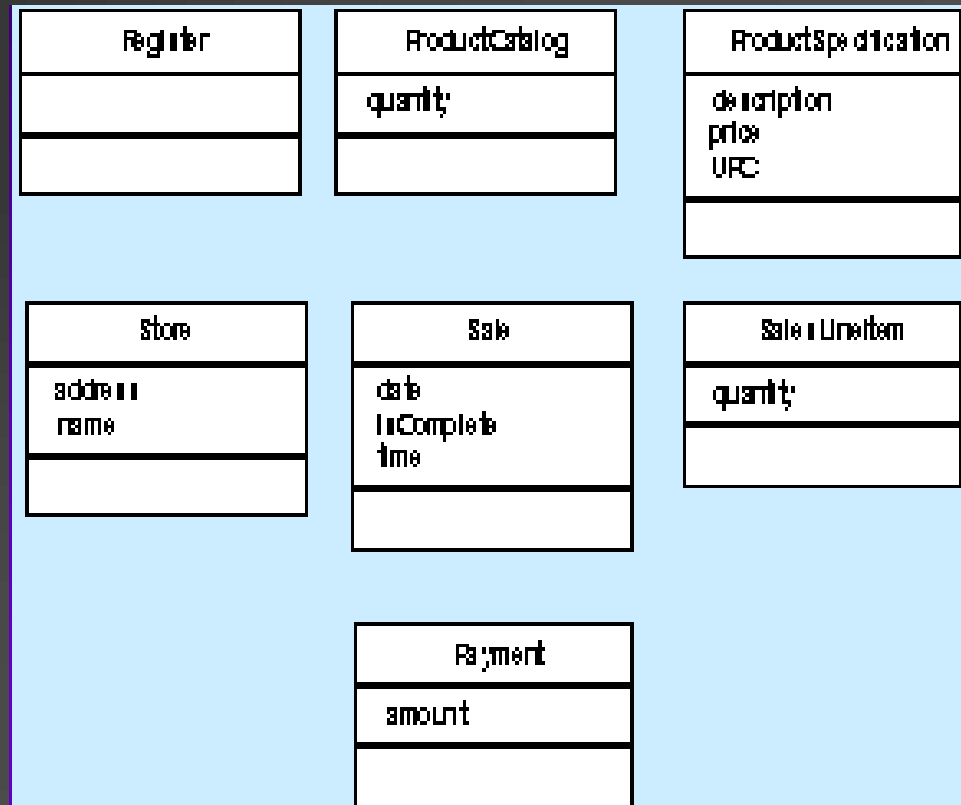
ProductSpecification

SalesLineItem



Software Classes

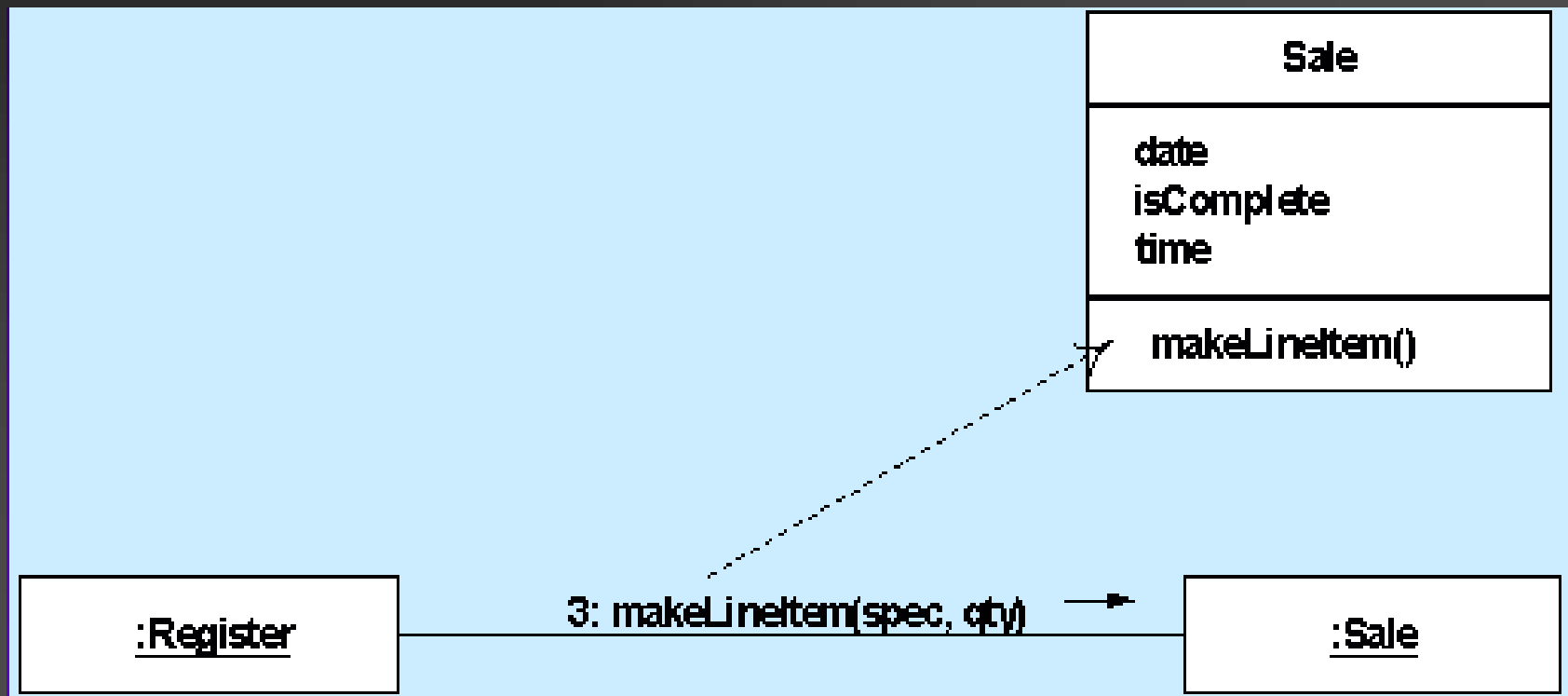
- Draw a class diagram for these classes.
- Include the **attributes** previously identified in the domain model.





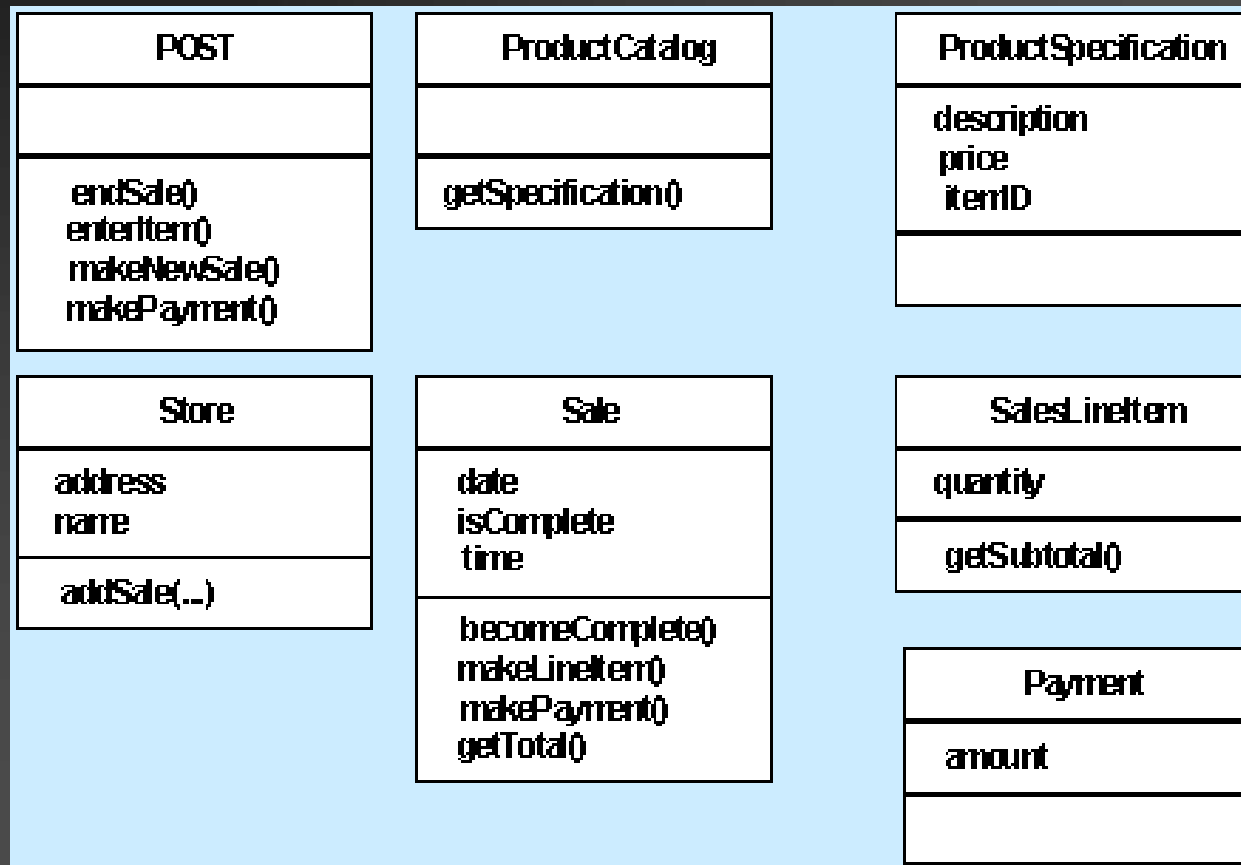
Add method name

- Method names from interaction diagrams





Methods.....





Method names: issues

- The following special issues must be considered with respect to method names:
 - Interpretation of the create() message.
 - Depiction of accessing methods.
 - Interpretation of messages to multi-objects.
 - Language-dependent syntax.
-



Method names: create

- The create message is the UML language independent form to indicate instantiation and initialization.
 - When translating the design to an OOPL it must be expressed in terms of its idioms for instantiation and initialization.
-

Method names:

Accessing methods



- Accessing methods are those which retrieve (accessor method - get) or set (mutator method) attributes.
- It is common idiom to have an accessor and mutator for each attribute, and to declare all attribute private (to enforce encapsulation).

Method names: Multi-objects

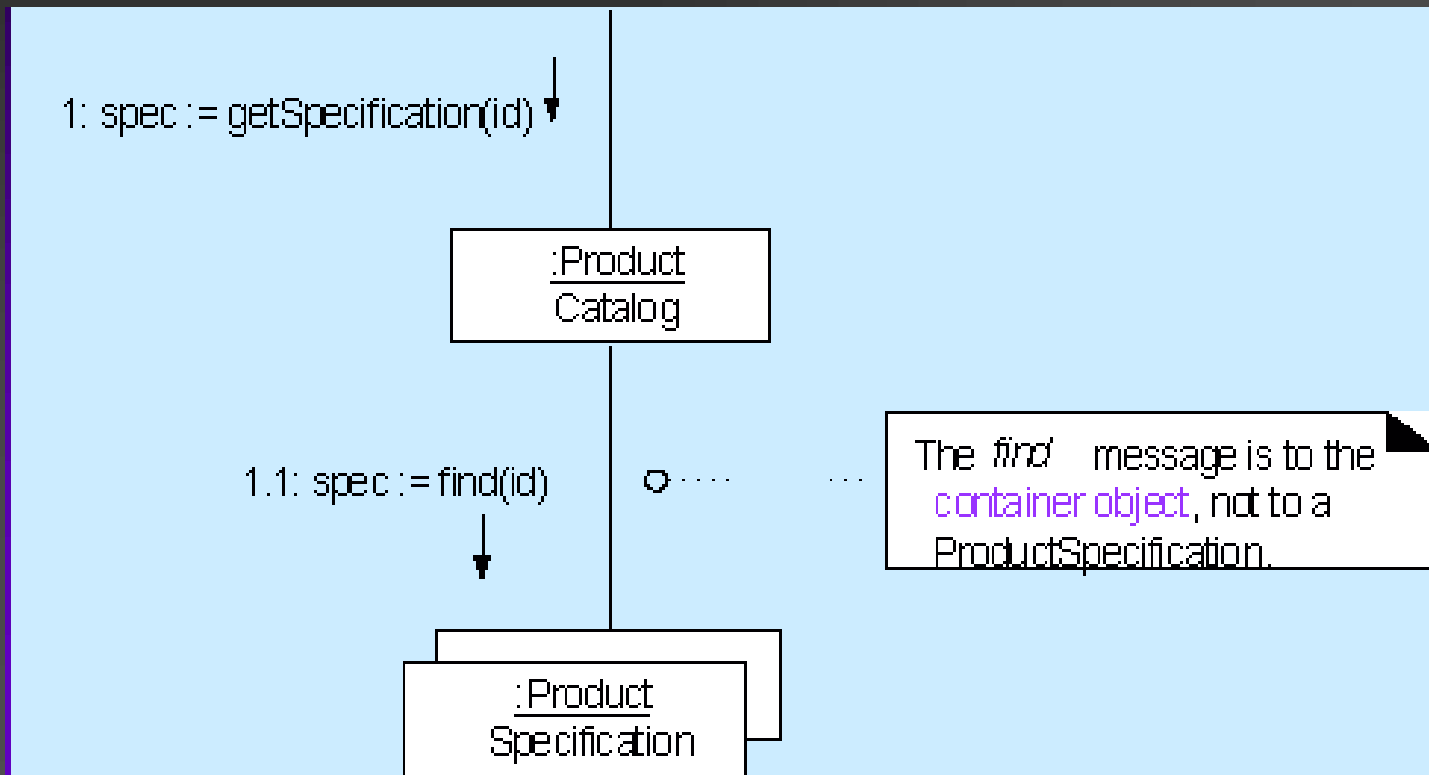


- A message to a multi-object is interpreted as a message to the container/collection object.



Message to a Multi-objects

- Java's map, C++'s map, Smalltalk's dictionary.



Method names: Language dependent syntax



- The basic UML format for methods:
 methodName(parameterList)

Adding more type information



- The type of the attributes, method parameters, and method return values may all optionally be shown.

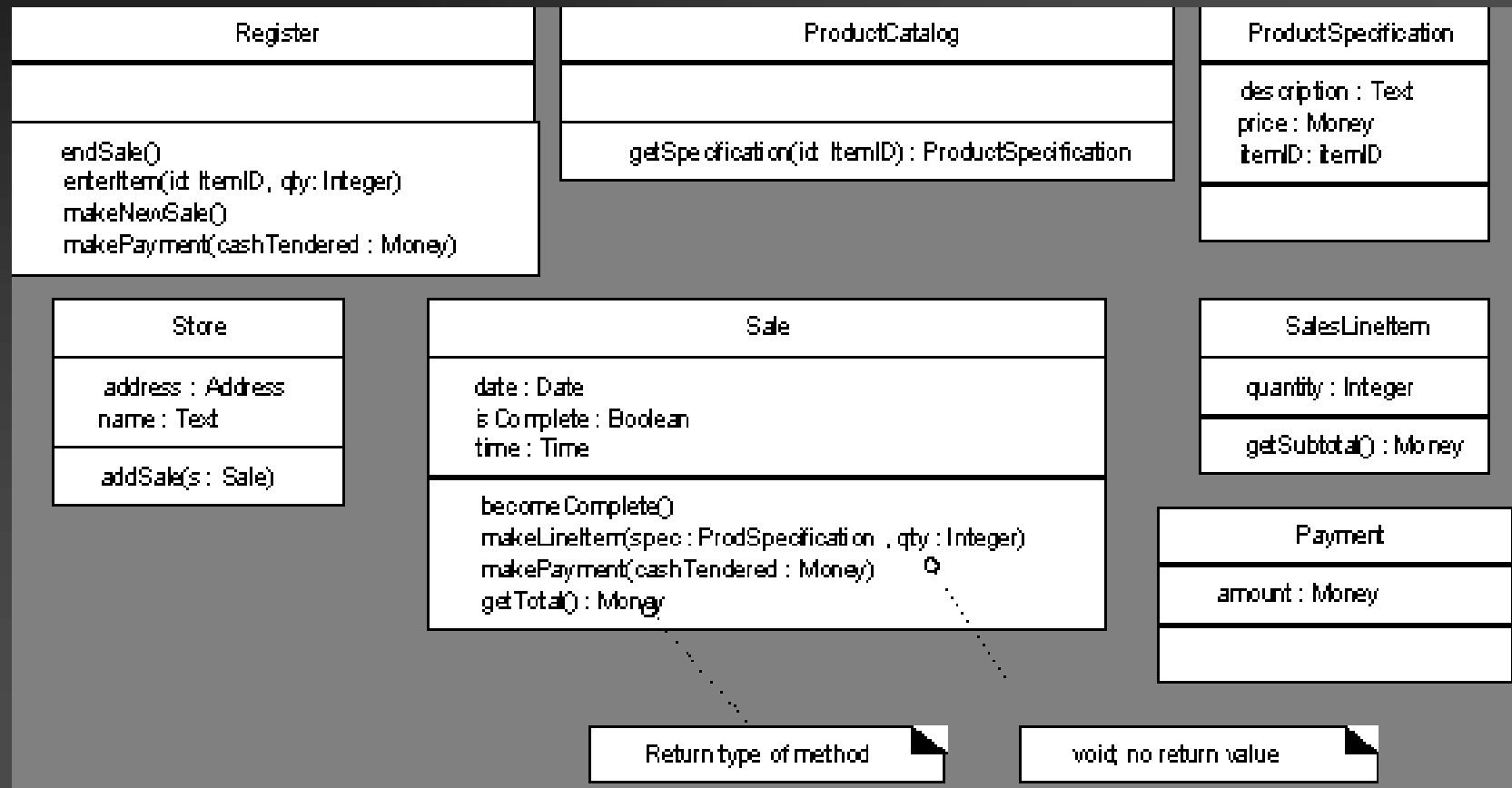


Continue.....

- The design class diagram should be created by considering the audience.
 - If it is being created in a CASE tool with automatic code generation, full and exhaustive details are necessary.
 - If it is being created for SW developers to read, exhaustive low-level detail may adversely effect the noise-to-value ratio.
-



Continue...

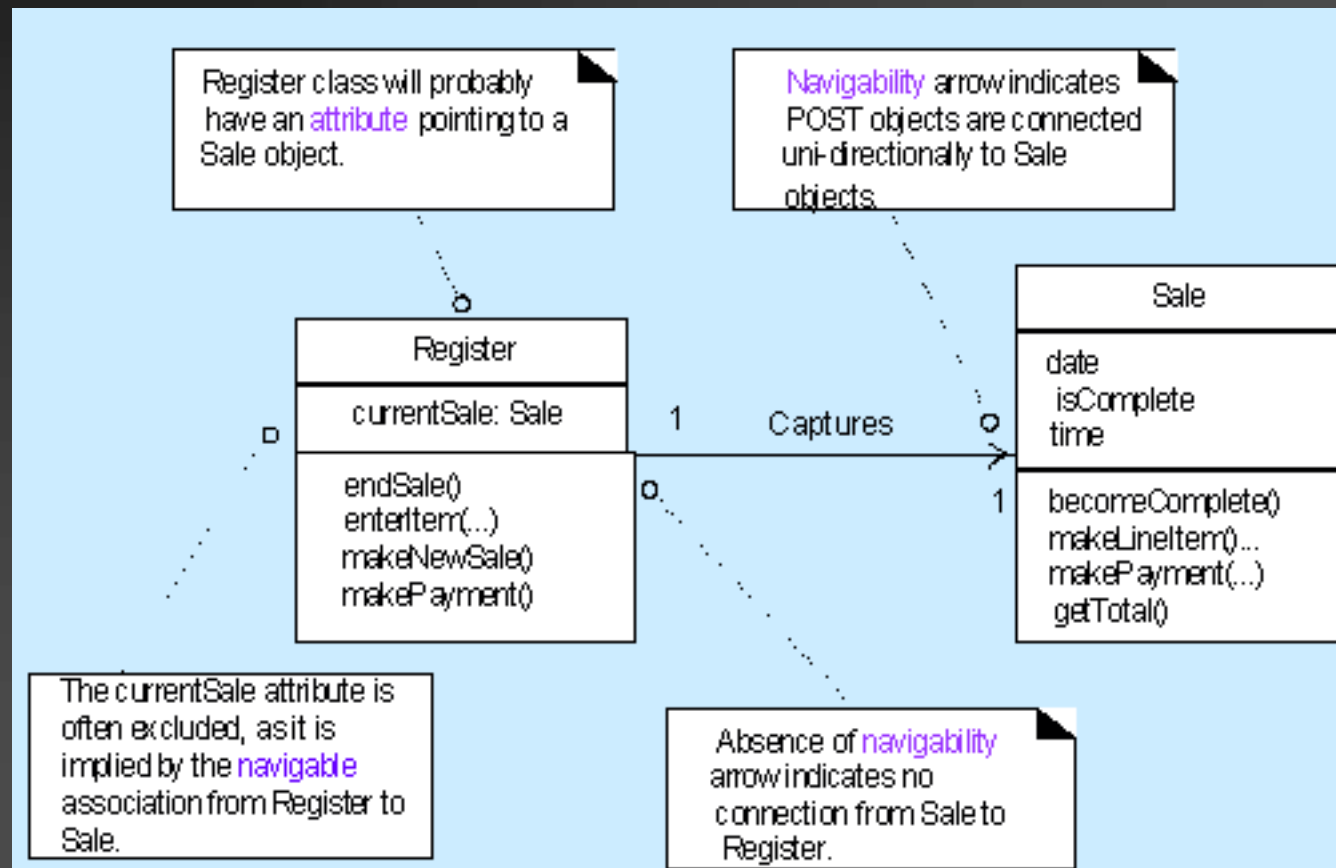


Adding associations and navigability



- Navigability is a property of the role which indicates that it is possible to navigate uni-directionally across the association from objects of the source to target class.
- Navigability implies visibility.
- Most, if not all, associations in design-oriented class diagrams should be adorned with the necessary navigability arrows.

Showing navigability, or attribute visibility.





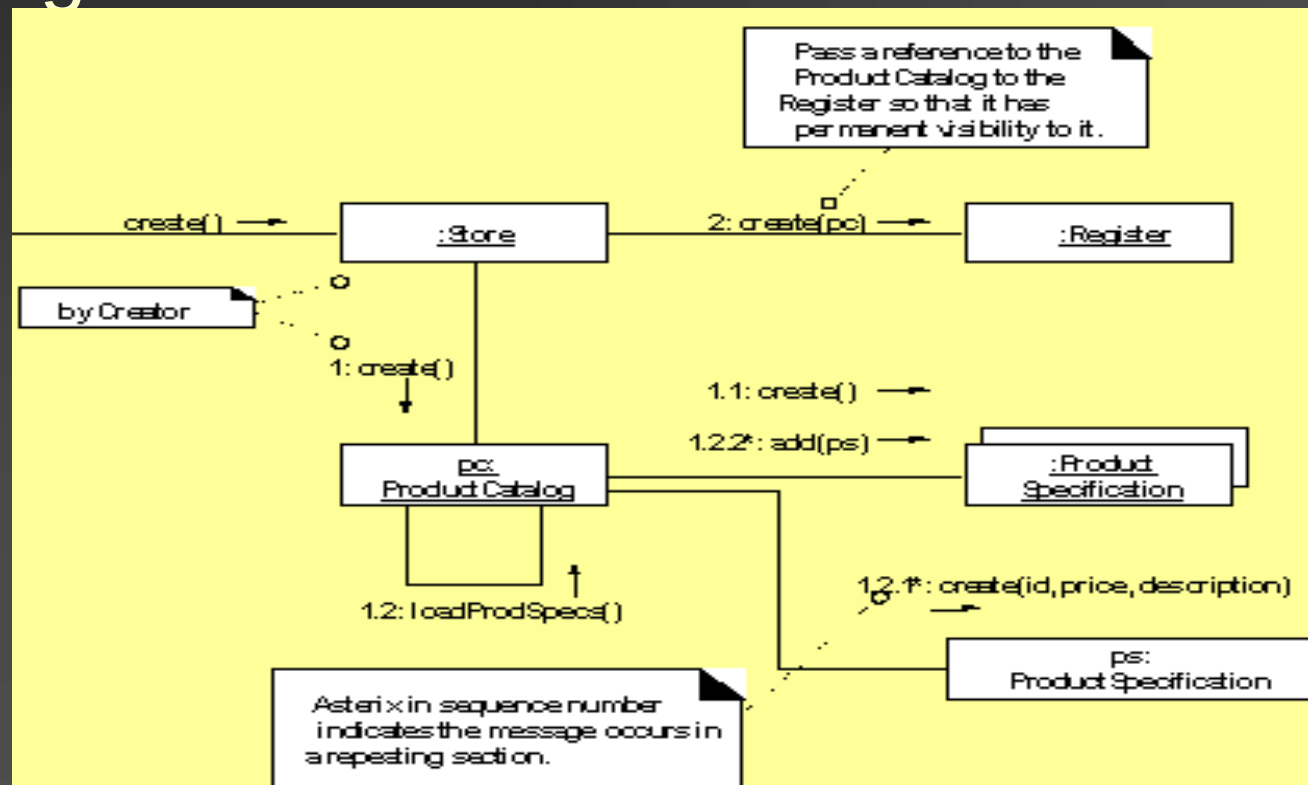
Association with navigability

- Define an association with a navigability adornment from A to B:
 - A sends a message to B.
 - A creates an instance B.
 - A needs to maintain a connection to B.

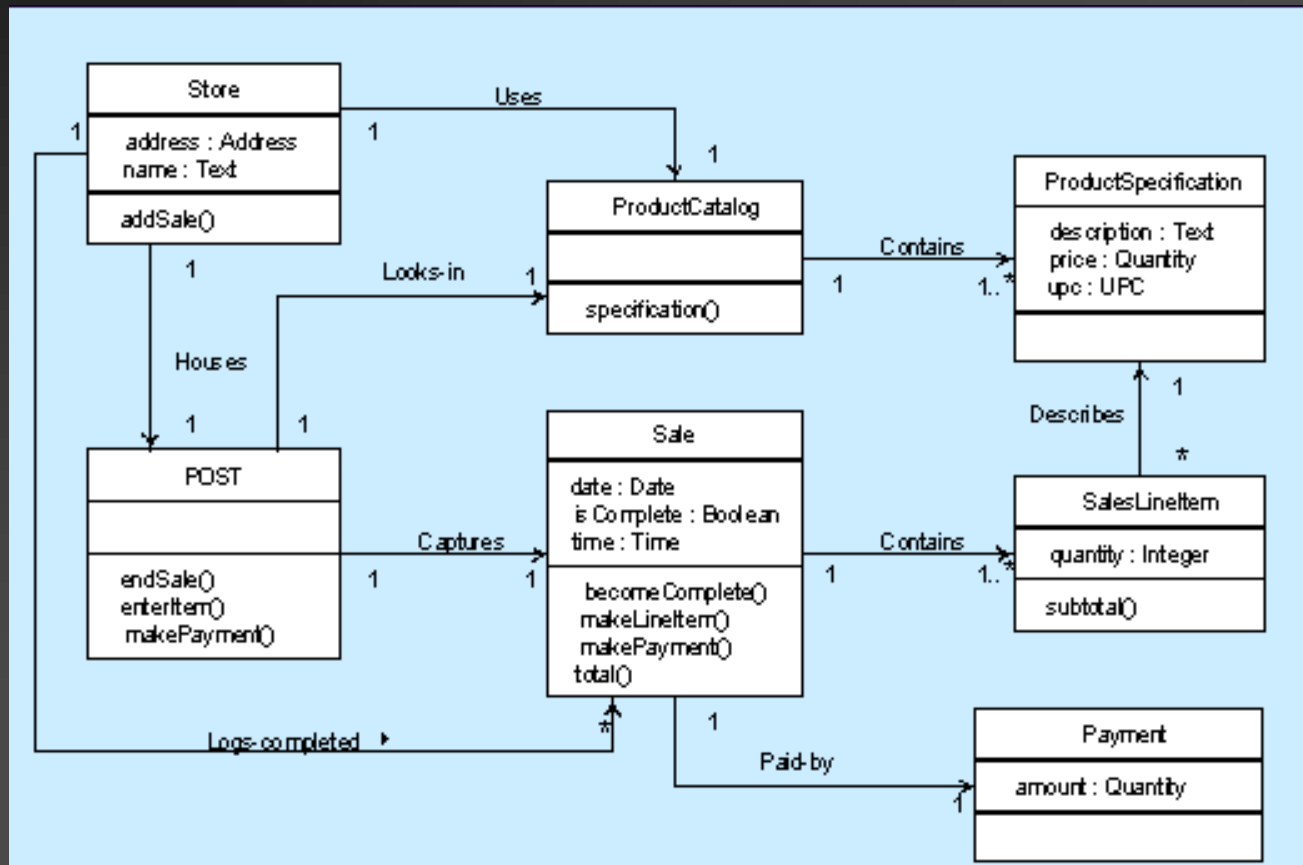


Navigability from CDs.

- Navigability is Identified from Interaction Diagrams.



Associations with navigability adornments



Adding dependency relationships



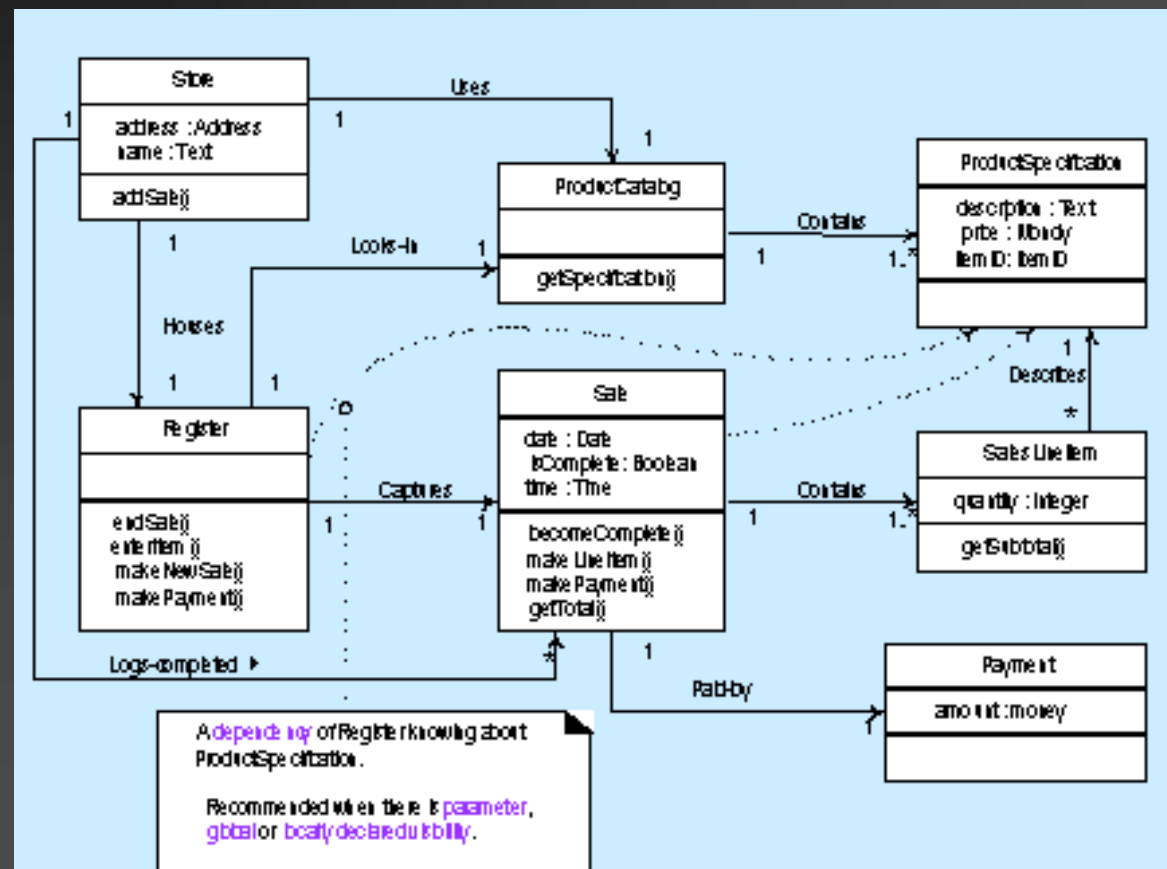
- Dependency relationship indicates that one element (of any kind, including classes, use cases, and so on) has knowledge of another element.
 - A dependency is a using relationship that states a change in specification of one thing may affect another thing that uses it, but not necessarily the reverse.



Continue...

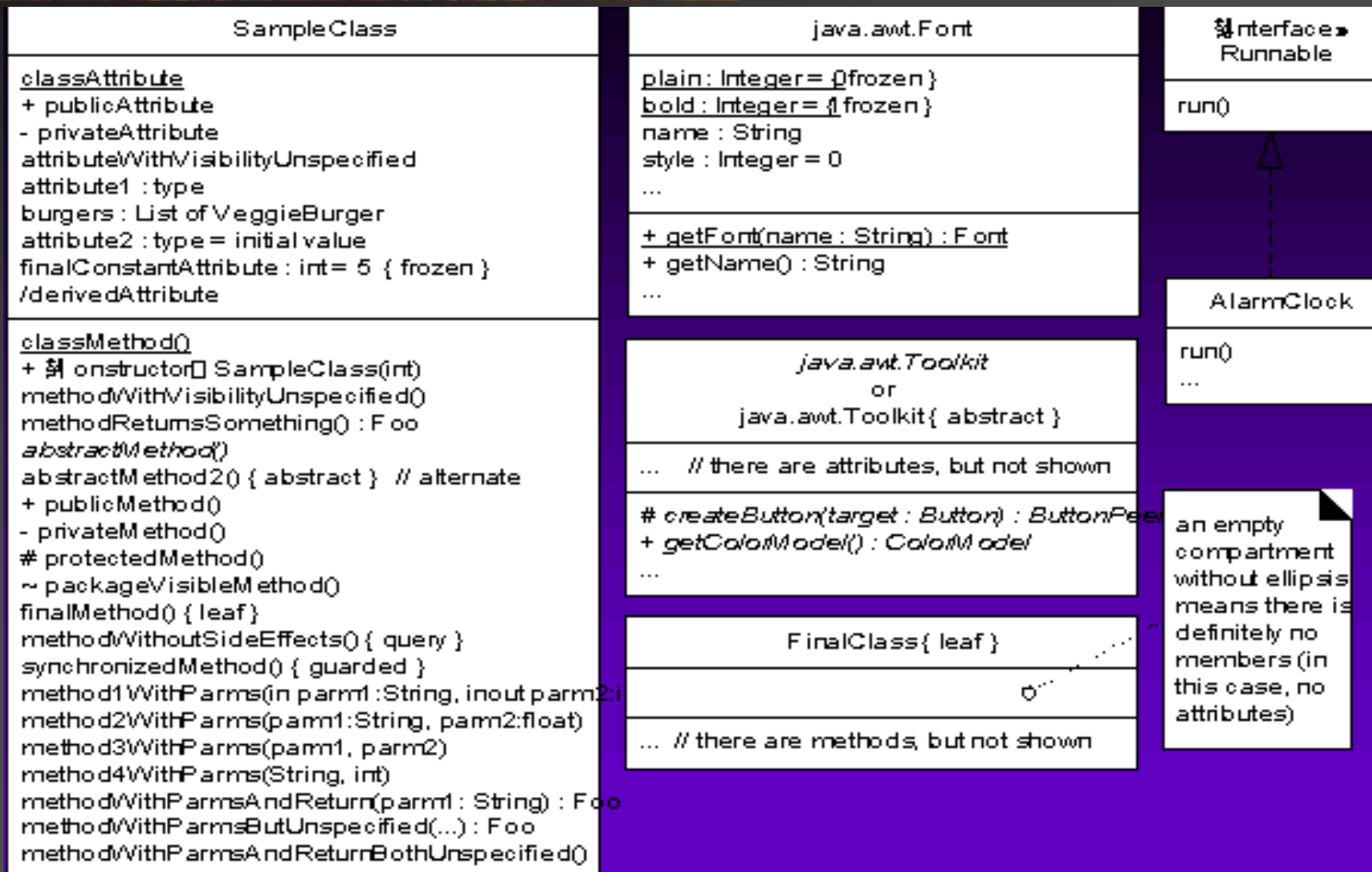
- The dependency relationship is useful to depict non-attribute visibility between classes.
 - Parameters
 - Global or local visibility
 - A dashed arrow line
 - A dashed directed line ----->

Dependency relationships non-attribute visibility

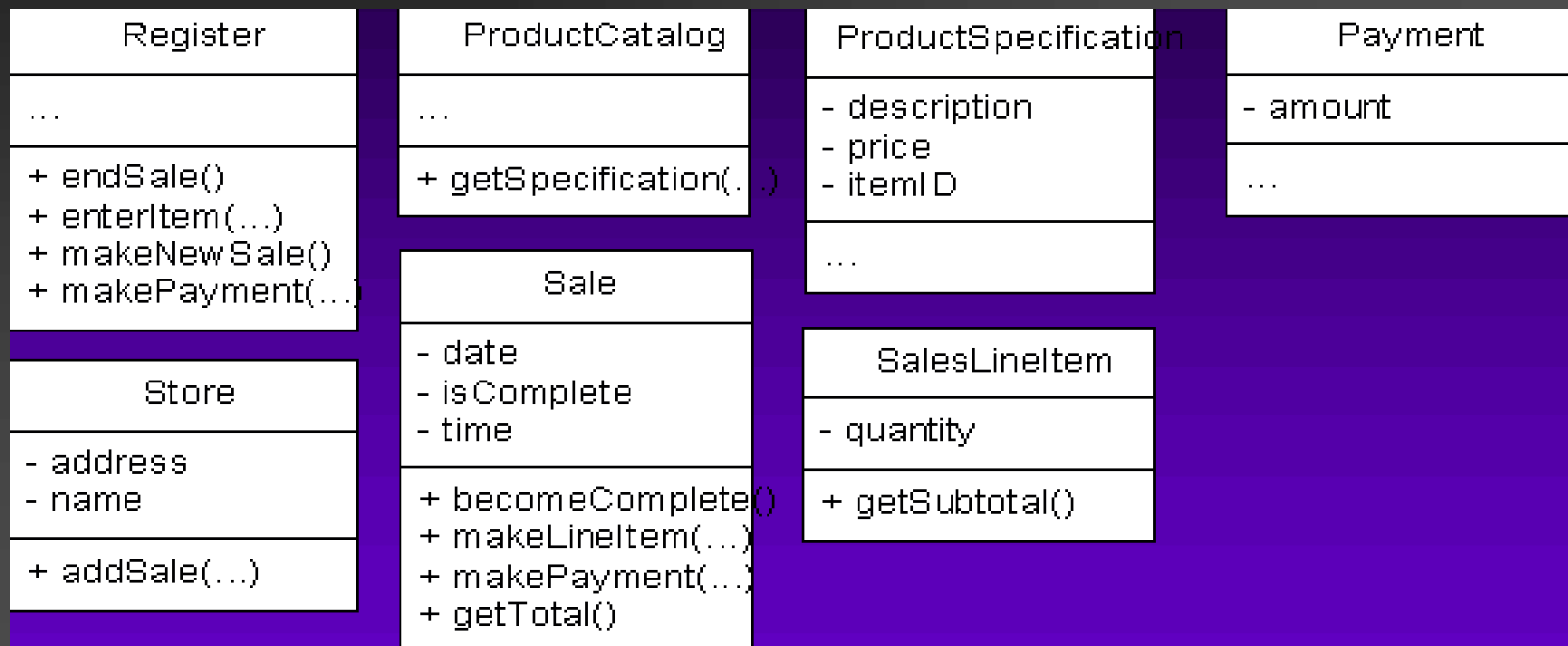




Notation for member details



Member details in the POS class diagram



Notation for method bodies in DCDs.



UML notation

A method body implementation may be shown in a UML note box. It should be placed vertically and signifies it is semantic influence (it is more than just a comment).

The syntax may be pseudo-code, or any language.

It is common to exclude the method signature (public void ...), but it is legal to include it.



```
{  
    ProductSpecification spec = catalog.getSpecification(id);  
    sale.makeLineItem(spec, qty);  
}
```

```
{  
    public void enterItem( id, qty )  
    {  
        ProductSpecification spec = catalog.getSpecification(id);  
        sale.makeLineItem(spec, qty);  
    }  
}
```

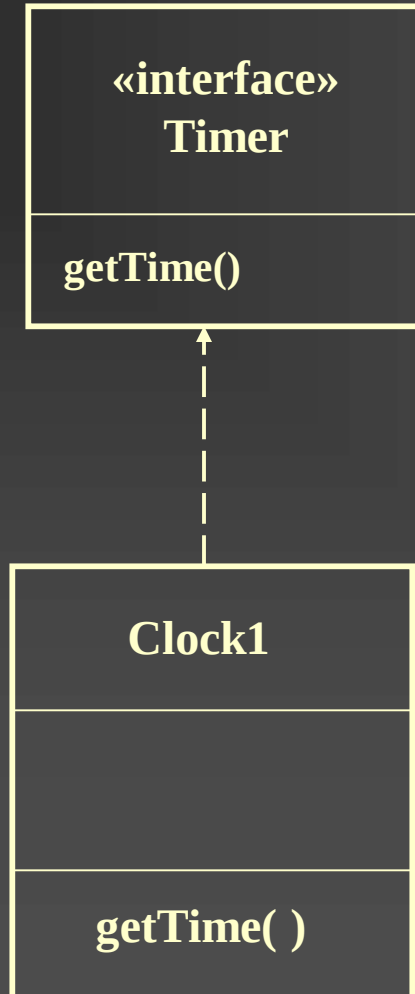


Interfaces

- The UML has several ways to show interface implementations. Since both the socket line notation and the dependency line notation used curved lines not common in drawing tools, most people use a dependency arrow and the «interface» stereotype.
-



Interface Example





UML stereotypes

- UML uses *stereotypes* encapsulated in guillemots (« and », *not* << and >>) to extend many diagram symbols.
- Guillemots can be found in the **latin-1** symbol set in Windows *Insert Symbol* command)
- Examples: «interface» «extends» «includes» «actor» «creates»



Association

- An association is a structural relationship that specifies that objects of one thing are connected to objects of another.
-



Navigation

- Navigability is a property of the role which indicates that it is possible to navigate unidirectionally across the association from objects of the source to target class.
- Navigability implies visibility.





Dependency

- A dependency is a using relationship, specifying that a change in the specification of one thing may affect another thing that uses it.





Generalization

- A generalization is a relationship between a general thing (called the super class or parent) and a more specific kind of that thing (called the subclass or child).
- is-a-kind-of relationship.





Aggregation

- Whole/part relationship
- Has-a relationship





Realization

- A realization is a semantic relationship between classifiers in which one classifier specifies a contract that another classifier guarantees to carry out.
- We use realization in two circumstances:
 - In the context of interfaces.
 - In the context of collaborations.





Phases

■ Inception

The Design Model and DCDs will not usually be started until elaboration because it involves detailed design decisions, which are premature during inception.



Continue...

■ Elaboration

- During this phase, DCDs will accompany the UC realization interaction diagrams; they may be created for the most architecturally significant classes of the design.
 - CASE tools can reverse-engineer (generate) DCDs from source code.
-



Continue...

■ Construction

- DCDs will continue to be generated from the source code as an aid in visualizing the static structure of the system.
-



UP Artifacts

